# PYTHON FOUNDATION CHALLENGE

**Week 8 — Revision, Progression & Stress Test**

## Objective

Week 7 pushed you through optimized math, matrices, and dictionary logic. Week 8 revisits classic problems with a higher bar, adds three new problems, and ends with one big stress test.

Questions 1–9 are standard practice: loops, strings, lists, matrices, and recursion. **Question 10 is the only question that brings Big O notation, hash maps, decorators, and recursion together.** It is reserved for students who completed Questions 6–8 last week.

## A Word Before You Start

By now you can write loops, functions, dictionaries, and recursive solutions. Week 8 raises the bar on testing and edge cases.

- Some questions are revision with a harder bar: the logic is familiar, the standards are higher.
- Some are new: struggling is the point.
- Bonus questions are optional. Do not touch them until the required ones are done.
- If you are stuck for more than 20 minutes on one question, move on, come back later, or write down the concept that is blocking you. **That note is more valuable than the answer.**

## Instructions

- Solve the problems independently. Do not look at previous solutions.
- Focus on producing a working solution first; clean it up afterward if time allows.
- No external libraries unless a question explicitly permits them.
- If you get stuck, identify the Python concept causing the problem rather than immediately searching for a solution.
- Test your code with the examples provided **and at least two additional cases of your own, including one edge case** (empty input, one element, duplicates, negative numbers).
- Prefer pure functions: `return` values from functions and keep `print()` outside them, unless the question says otherwise.

## Week 8 at a Glance

| Theme | Questions |
|---|---|
| Search, sort, and one-pass logic | 1, 3, 6 |
| String and number logic | 2, 4 |
| Recursion | 5, 9 |
| Nested lists | 7 |

| Theme | Questions |
|---|---|
| Mini project (state and edge cases) | 8 |
| Stress test: Big O, hash maps, decorators, recursion (Questions 6–8 group only) | 10 |

**The progression:** searching → string encoding → one-pass logic → number conversion → recursion → sorting → building a triangle → a small game → backtracking → the integrated stress test.

---

# Part A — Revision (Higher Bar)

These revisit topics from earlier weeks. The logic may be familiar. The standard is not: this time you must test more carefully and handle edge cases.

## 1. Binary Search (Revision: Count Your Steps)

**Suggested time:** 20–25 minutes

### Problem

Given a **sorted** list of integers and a target, return the index of the **first (leftmost) occurrence** of the target, or `-1` if it is not present. Also return how many comparisons your search made.

### Examples

```
nums = [1, 3, 3, 3, 5, 8], target = 3  → (1, comparisons)
nums = [1, 3, 5, 8],       target = 4  → (-1, comparisons)
nums = [],                 target = 7  → (-1, 0)
```

### Constraints

- Use a `while` loop with `low` and `high` pointers. No recursion in the main version.
- No `.index()`, no `in`, no `.count()`.
- Must find the **leftmost** match when duplicates exist. Finding any match is not enough.
- The function `binary_search(nums, target) -> tuple[int, int]` must be pure: no printing inside.
- Do not import libraries.

### Bonus

Write a recursive version of binary search and check that it returns the same answers as your loop version on at least five test cases.

**What this gauges:** Can you shrink the problem each step instead of scanning everything? Can you handle duplicates and empty lists?

---

## 2. Run-Length Encoding & Decoding (Revision: Round Trip)

**Suggested time:** 25–30 minutes

### Problem

Write two functions: `rle_encode(text)` and `rle_decode(encoded)`. Decoding must reverse encoding perfectly, including counts with more than one digit.

### Examples

```
rle_encode("aaabbc")        → "a3b2c1"
rle_encode("aaaaaaaaaaab") → "a12b1"
rle_decode("a12b1")         → "aaaaaaaaaaab"
rle_encode("")              → ""
```

### Constraints

- Input text contains **letters only** (no digits), so the encoded form is unambiguous.
- Use `while` loops and index tracking. No `itertools`, no `re`.
- `rle_decode` must read a **whole number** after each letter (e.g., `12`), not just one digit.
- Build the result with string concatenation or a list joined at the end.
- Round-trip check: for every test string `s`, `rle_decode(rle_encode(s)) == s` must be `True`. Print the result of at least four round-trip checks.
- Do not import libraries.

### Bonus

Add a function `compression_ratio(text)` that returns `len(encoded) / len(text)`. Find one input where RLE makes the string *longer* and explain why in one sentence.

**What this gauges:** Look-ahead and index control, multi-digit parsing, and testing a function against its own inverse.

---

## 3. Maximum Subarray (Revision: Brute Force vs Single Pass)

**Suggested time:** 25–30 minutes

### Problem

Given a list of integers (positive and negative), find the contiguous slice with the largest sum. Return the sum **and** the start and end indices.

### Examples

```
[-2, 1, -3, 4, -1, 2, 1, -5, 4] → sum 6, indices (3, 6)   # [4, -1, 2, 1]
[5]                              → sum 5, indices (0, 0)
[-3, -1, -2]                     → sum -1, indices (1, 1)
```

### Constraints

- Write **two** functions:
    a. `max_subarray_brute(nums)`: checks every possible slice (nested loops are allowed here).
    b. `max_subarray_fast(nums)`: a **single pass** with no nested loops.
- Both must return identical answers on every test.
- No `max()`, `min()`, `sum()`, or slicing to compute sums.
- All-negative lists must return the single largest element, not `0`.
- Do not import libraries.

### Hint

At each position, ask: *is it better to extend the running sum, or to start fresh here?*

### Bonus

Write a test loop that runs both versions on at least five lists of your own (including an empty list) and prints `Match` or `Mismatch` for each.

**What this gauges:** Can you replace brute force with a single-pass insight, and *prove* the two agree?

---

## 4. Integer to Roman (Revision: The Inverse Problem)

**Suggested time:** 20–25 minutes

### Problem

Convert an integer from 1 to 3999 into a Roman numeral. This is the reverse of Roman to Integer.

### Examples

```
3    → "III"
58   → "LVIII"
1994 → "MCMXCIV"
3999 → "MMMCMXCIX"
```

### Constraints

- Use a greedy approach with an **ordered list of (value, symbol) pairs** that includes the subtractive forms (`900 → CM`, `400 → CD`, `90 → XC`, `40 → XL`, `9 → IX`, `4 → IV`).
- Use a `while` loop. Do not hardcode answers.
- Return `""` or raise a clear error message for numbers outside 1–3999 (your choice, but be consistent and say which).
- Do not import libraries.

### Bonus

Write a check loop that converts every integer from 1 to 3999 to Roman and back using your Week 7 `roman_to_integer`. Print `All round trips passed` or the first failure.

**What this gauges:** Greedy thinking, ordered data, and verifying a function by pairing it with its inverse.

---

## 5. Tower of Hanoi (Revision: Recursion as a Recipe)

**Suggested time:** 25–30 minutes

### Problem

Write `hanoi(n, source, target, helper)` that returns the **list of moves** needed to move `n` disks from `source` to `target`, never placing a larger disk on a smaller one.

### Examples

```
hanoi(1, "A", "C", "B") → [("A", "C")]
hanoi(3, "A", "C", "B") → [("A","C"), ("A","B"), ("C","B"), ("A","C"), ("B","A"),
("B","C"), ("A","C")]
```

### Constraints

- Must be **recursive**. No loops inside `hanoi`.
- Must be a **pure function** that returns the moves. Do not print inside it.
- Print the number of moves for `n = 1` to `10` from *outside* the function. What pattern do you see?
- Do not import libraries.

### Bonus

Prove (with a loop outside the function) that `len(hanoi(n, ...)) == 2**n - 1` for `n` from 1 to 12.

**What this gauges:** Can you trust a recursive call to solve the smaller problem? Can you spot the pattern in the move counts?

---

## 6. Bubble Sort (Revision: Best Case vs Worst Case)

**Suggested time:** 20–25 minutes

### Problem

Implement bubble sort that sorts a list **in place** and returns the number of comparisons and swaps it performed.

### Examples

```
[5, 1, 4, 2, 8] → sorted [1, 2, 4, 5, 8], with counts
[1, 2, 3, 4, 5] → already sorted: your optimized version should stop after ONE pass
[]              → [] with 0 comparisons and 0 swaps
```

### Constraints

- No `sort()`, `sorted()`, `min()`, `max()`.

- Implement the **early-exit optimization**: if a full pass makes no swaps, stop immediately.
- Each pass should check one fewer pair than the last (the largest items have already settled at the end).
- Show with comparison counts that your function makes fewer comparisons on an already-sorted list than on a reverse-sorted list of the same size.
- Do not import libraries.

#### Bonus

Add a `descending=True` parameter that sorts largest to smallest.

**What this gauges:** Can you measure your own code, and do you understand that one algorithm can have different costs depending on its input?

---

## Part B — New Questions: Progression

These are new problems built around nested lists, a small game, and backtracking. Loops are allowed unless a question says otherwise.

### 7. Pascal's Triangle

**Suggested time:** 25–30 minutes

#### Problem

Build the first `n` rows of Pascal's Triangle. Each row starts and ends with `1`, and every other number is the sum of the two numbers directly above it in the previous row.

#### Examples

```
n = 5 → [[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]
n = 1 → [[1]]
n = 0 → []
```

Printed as a triangle (`n = 5`):

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

#### Constraints

- Write a pure function `pascal_rows(n) -> list[list[int]]` that returns the rows. No printing inside it.
- Build each row **from the previous row** using nested loops. No factorials, no formulas, no `math`.
- No list comprehensions.
- Print the triangle from **outside** the function, adding the leading spaces yourself (no `.center()`).
- Do not import libraries.

### Bonus

Write `pascal_row(k)` that returns only row `k` (counting from 0) while keeping just one list in memory at a time.

**What this gauges:** Can you build each step from the one before it? Are your nested loops and indexes correct at the edges of every row?

---

## 8. Hangman (Mini Project)

**Suggested time:** 35–40 minutes

### Problem

Build the logic for a game of Hangman. A word is hidden. The player guesses one letter at a time and is allowed a limited number of wrong guesses.

Write `play_hangman(word, guesses, max_attempts)` that plays through a list of guesses and returns a tuple `(status, display, wrong_guesses)`:

- `status` is `"won"`, `"lost"`, or `"playing"`.
- `display` is the word with unguessed letters shown as `_` (no spaces).
- `wrong_guesses` is a list of the wrong letters, in the order they were guessed.

### Rules

- Every guess is a single lowercase letter.
- A correct guess reveals every matching position.
- A wrong guess uses up one attempt.
- Repeating a guess (right or wrong) does nothing and costs no attempt.
- The game ends the moment all letters are revealed (`"won"`) or the wrong guesses reach `max_attempts` (`"lost"`). Any remaining guesses are ignored.
- If the guesses run out before either happens, the status is `"playing"`.

### Examples

```
play_hangman("apple", ["a", "z", "p", "l", "e"], 3) → ("won", "apple", ["z"])
play_hangman("python", ["a", "b", "c", "d"], 3)     → ("lost", "______", ["a", "b",
"c"])
play_hangman("python", ["p", "p", "y"], 5)          → ("playing", "py____", [])
play_hangman("a", [], 3)                            → ("playing", "_", [])
```

### Constraints

- Write `play_hangman` as a pure function. No `print()` or `input()` inside it.
- Use loops and conditionals. Track guessed letters in a list.
- Do not hardcode answers. Test with at least three words of your own.
- Do not import libraries.

### Bonus

Write a small `main()` outside the function that uses `input()` to ask for one letter at a time and shows the display and remaining attempts after each guess, until the game ends. Reuse your game logic rather than rewriting it.

**What this gauges:** Can you track state across many steps, handle repeats and early stops, and keep game logic separate from input and output?

---

## 9. Subsets and Permutations (Recursion & Backtracking)

**Suggested time:** 30–35 minutes

### Problem

Write two recursive functions:

1. `subsets(items)`: return all subsets (the power set) of a list.
2. `permutations(items)`: return all possible orderings of a list of distinct items.

### Examples

```
subsets([1, 2, 3])
→ [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]   # any order is fine

permutations([1, 2, 3])
→ [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]   # any order is fine

subsets([])      → [[]]
permutations([]) → [[]]
```

### Constraints

- Both must be **recursive**. No `itertools`.
- Neither function may modify the list passed in.
- Return a fresh list of fresh lists every time.
- Test with `n` from 0 to 8 and print how many results each function returns. What pattern appears? ($2^n$ and `n!`)
- Do not import libraries.

### Hint

For subsets: for each item you either **take it** or **skip it**. For permutations: pick one item to go first, then permute whatever remains.

### Bonus

Write `subsets_with_sum(items, target)` that returns only the subsets that add up to `target`.

**What this gauges:** Can you think in "choices" instead of loops? Can you build up a result on the way back out of recursion?

---

# Part C — Stress Test

## 10. The Performance Lab (Stress Test)

**Group:** Students who completed Questions 6–8 last week (Anagram Checker, Roman to Integer, Matrix Rotation). **Suggested time:** 60–75 minutes **Tests:** Big O notation, hash maps, decorators, and recursion, all in one program.

This question is deliberately big. You are expected to plan before you code. Read the whole question first, sketch your functions on paper, then build one part at a time. If you finish Parts 1–4 cleanly, you are working at a strong level for this stage.

**Big O in one minute.** Big O describes how the work your code does *grows* as the input grows. It ignores small details and focuses on the shape of the growth.

- O(1): constant. Same work no matter the input size (a dictionary lookup).
- O(log n): the input is halved each step (binary search).
- O(n): one pass through the data.
- O($n^2$): a loop inside a loop over the same data.
- O($2^n$) and O(n!): the work doubles (or worse) with every extra item. Fine for tiny inputs, impossible for large ones.

Whenever this question asks for a time or space complexity, write it as a short comment above your function with one sentence explaining why.

### Part 1: Recursion

Write `count_ways(amount, coins)` that returns the **number of different combinations of coins** that make up `amount`. Order does not matter: `1+2` and `2+1` count as one way. Coins can be reused.

```
count_ways(5, [1, 2, 5])         → 4
count_ways(10, [2, 5, 3, 6])     → 5
count_ways(0, [1, 2])            → 1     # one way to make nothing: use no coins
count_ways(3, [2])               → 0
count_ways(100, [1, 5, 10, 25, 50]) → 292
```

- Must be **recursive** (use a helper such as `count_ways(amount, coins, index)` if you need one).
- The **naive** version must make no use of caching. Run it on `count_ways(100, [1, 5, 10, 25, 50])` and notice how much work it does.

### Part 2: Decorator

Write a decorator factory `memoize(max_size=None)` that caches results.

- The cache is a **dictionary stored in the closure**, with the function arguments (as a tuple) as the key.

- The wrapper must expose these as attributes: `wrapper.hits`, `wrapper.misses`, and `wrapper.calls` (hits + misses).
- If `max_size` is given and the cache would grow beyond it, **remove the oldest entry** (the one inserted first) before storing the new one. Python dictionaries remember insertion order, so you can find the oldest key without any libraries.
- If `max_size` is `None`, the cache is unlimited.
- Must work with `@memoize()` and `@memoize(max_size=50)`.
- No global variables.

Apply it so that **the same recursive logic** from Part 1 becomes fast. Because your `count_ways` takes a list (unhashable), you must make it hashable: convert the coins to a `tuple` before caching. Explain in a comment why that conversion is necessary.

### Part 3: Hash Maps

Write two O(n) functions using dictionaries (no `sort`, no `sorted`, no `set`, no `collections`):

1. `longest_consecutive(nums)`: return the length of the longest run of consecutive integers in the list, in any order, without sorting.

```
longest_consecutive([100, 4, 200, 1, 3, 2]) → 4      # 1, 2, 3, 4
longest_consecutive([])                      → 0
longest_consecutive([5, 5, 5])               → 1
longest_consecutive([0, -1, 1, 2, -2])       → 5      # -2, -1, 0, 1, 2
```

*Hint: use a dictionary as a fast lookup table. Only start counting from a number whose predecessor is **not** in the table.*

2. `count_pairs(nums, target)`: return how many pairs of positions `(i, j)` with `i < j` satisfy `nums[i] + nums[j] == target`. Use a frequency dictionary in a single pass.

```
count_pairs([1, 5, 7, -1, 5], 6) → 3      # (1,5), (1,5), (7,-1)
count_pairs([1, 1, 1], 2)        → 3
count_pairs([], 5)               → 0
```

### Part 4: Instrumentation and Big O Report

Write a `main` section (outside the analysis functions) that prints a short report:

1. Define `fib` naively (recursive, no cache) with a call counter, and a memoized version using your `memoize`. For `n = 10, 20, 25`, print the number of function calls each version made. For `fib(35)` and `fib(90)`, print the memoized result and `fib.misses`. Expected values: `fib(35) = 9227465`, `fib(90) = 2880067194370816120`.
2. Print `hits`, `misses`, and `calls` for memoized `count_ways(100, [1, 5, 10, 25, 50])`. Confirm it prints `292`.
3. Run `memoize(max_size=3)` on a function and call it with 5 different arguments. Show the cache never holds more than 3 entries and that an evicted value becomes a *miss* again when requested.
4. Print a table (formatted with string logic, no libraries) with the columns: **function name | time complexity | space complexity**, for every function you wrote in this question, **before** and **after** memoization where relevant.

### Part 5: Written Analysis (as comments)

Answer these in comments at the bottom of your file. Short, specific answers.

1. What is the time complexity of naive `fib(n)`? Of memoized `fib(n)`? Explain what the memoized version avoids doing.
2. Why does memoizing `longest_consecutive` give almost no benefit?
3. Why does a dictionary lookup take O(1) on average, and what could make it slower in the worst case?
4. Give one situation where memoizing would be a **bad idea** (think memory, or functions with side effects).
5. For `memoize(max_size=3)`, what is the space complexity? What is the tradeoff of choosing a very small `max_size`?

### Constraints (whole question)

- No external libraries (not even `functools`; write your own decorator).
- Do not hardcode answers.
- Analysis functions must be **pure** and return values. Printing lives in the `main` section only.
- Do not modify any list passed into a function.
- Handle edge cases: empty lists, zero amounts, negative numbers, duplicates.

### Hard Bonus

1. Rewrite `count_ways` **iteratively** using a list of size `amount + 1` (bottom-up). Compare its space and time complexity to the memoized recursive version. Which would you choose for `amount = 10000`, and why? (Hint: think about Python's recursion limit.)
2. Make `memoize` evict the **least recently used** entry instead of the oldest inserted. When a cached key is hit, move it to the "newest" position.

**What this gauges:**

- Recursion: can you express a counting problem as take/skip choices?
- Big O: can you predict and *measure* growth, not just recite it?
- Hash maps: can you design keys and lookups that remove nested loops?
- Decorators: can you build one with its own arguments, state in a closure, and attributes on the wrapper?
- Integration: can you combine all four and explain the result in plain words?

---

## Progression

Binary search → string encoding with look-ahead → one-pass logic → greedy number conversion → recursion on moves → sorting with counted work → building a triangle row by row → a small game with state → backtracking → one stress-test program (Q10 only) that makes you recurse, cache, hash, decorate, and explain the Big O of all of it.

The through-line is **careful testing and clear thinking**. Working code is the floor; handling edge cases and explaining your choices is the goal.

## Time Guide

| Group | In class (9:30 AM – 12:45 PM) | Suggested time | Take-home |
|---|---|---|---|
| Everyone | Q1, Q3, Q5, Q7, Q8, Q9 | 2 hrs 40 min – 3 hrs 10 min | Q2, Q4, Q6 |
| Q6–8 group | Q10, Q1, Q3, Q5, Q9 | 2 hrs 40 min – 3 hrs 15 min | Q2, Q4, Q6, Q7, Q8 |

The session runs 3 hrs 15 min, so everything in the "In class" column fits with a little slack. Finish each question inside its suggested time. If you pass the top of the range, write down what blocked you and move on. Take-home questions are due at the next session. For the Q6–8 group, prioritize Q10 Parts 1–4 in class and finish the Part 5 written answers at home if time runs short. Doing every question would take 3 hrs 45 min – 4 hrs 30 min (Q1–9), or 4 hrs 45 min – 5 hrs 45 min including Q10.

## Self-Assessment Guide

| Question | Concept being tested | What it tells you |
|---|---|---|
| 1. Binary Search | Halving, leftmost match | Can you shrink the problem each step? |
| 2. Run-Length Encoding | Index control, multi-digit parsing | Can you build and test an inverse? |
| 3. Max Subarray | Brute force vs single pass | Can you spot the one-pass insight? |
| 4. Integer to Roman | Greedy ordered data | Can you reverse a problem you solved before? |
| 5. Tower of Hanoi | Recursion | Do you trust the recursive call? |
| 6. Bubble Sort | Early exit, counting work | Can you measure your own code? |
| 7. Pascal's Triangle | Nested loops, building from the previous row | Can you build each step from the last? |
| 8. Hangman | State tracking, edge cases | Can you separate game logic from input and output? |
| 9. Subsets & Permutations | Backtracking | Can you think in choices instead of loops? |
| 10. Performance Lab (Q6–8 group) | Big O, hash maps, decorators, recursion | Can you combine and justify everything? |

## How to Read Your Results

- **7–9 required questions solved cleanly** → Strong. You are testing carefully and handling edge cases.
- **4–6 solved** → Solid fundamentals. Keep practicing the questions that blocked you.
- **1–3 solved** → Useful information, not failure. Note which concept blocked you (recursion, nested loops, string handling, or state tracking) and bring that list to the session.
- **Q9 solved** → You can think recursively and build results on the way back out.
- **Q10 Parts 1–4 solved** → You can integrate recursion, hashing, decorators, and complexity analysis in one program.
- **Q10 Part 5 answered well** → You can explain your code to someone else. That is the skill that matters most.

## A Note on "Gauging"

This challenge is a gauge, not an exam. The point is to find out which ideas feel natural and which need another pass. Struggling is data, not failure. If you finish half of this and can name exactly which concept blocked you on the rest, you have learned more than someone who copied answers to all ten.

**Write down your blockers. Bring them to the next session. That list is the real deliverable.**

## Notes for the Instructor

| Design choice | Reason |
|---|---|
| Q1–Q9 are standard practice with no Big O, hash map, or decorator requirements | Keeps the main set accessible. Those topics are saved for Q10. |
| Revision topics come from quests not yet revisited (search, sort, RLE, Kadane, Roman inverse, Hanoi) | Covers gaps in the completed-exercises list while keeping the "revision with a harder bar" format. |
| Q7 Pascal's Triangle | Nested loops that build each row from the previous one; good warm-up for matrix work. |
| Q8 Hangman | A mini project from the exercise list not yet done. Practices state tracking, edge cases, and separating logic from input and output. |
| Q9 is the first backtracking problem | Natural step after Hanoi and flatten/group. |
| Q10 is the only question combining Big O, hash maps, decorators, and recursion | Reserved for the Q6–Q8 group as a stress test. Parts 1–4 build the program; Part 5 checks understanding. |
| Q10 expected values | `count_ways(100, [1,5,10,25,50]) = 292`, `fib(35) = 9227465`, `fib(90) = 2880067194370816120`. |

**End of Week 8 Challenge**